

Exercise 1

Function A - Common Value

1. Input size : $m = \text{first.length}$ and $n = \text{second.length}$
2. Primitive operation: **first[i] == second[j]**, it loops
3. Cost function: return true is executing early which mean that if $\text{first}[0] == \text{second}[0]$ then the best case can be a simple comparison. If the arrays doesn't share any values, every pair will get compared, which means outer loop will run m times and with `isync` the inner loop will run n times:

$$T(m, n) = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} 1 = \sum_{i=0}^{m-1} n = mn$$

4. Bound: $O(mn)$
5. Space: Auxiliary space is $O(1)$, and the result is a single boolean $O(1)$

Function B - Increasing Triples

1. Input size: $n = \text{values.length}$
2. Primitive operation: Is to evaluate the **if** condition.
3. Cost function: because loop binds the force $i < j < k$, with each index are between 0 and $n-1$. Since combination are only run once, in increasing order.

$$T(n) = \binom{n}{3} = \frac{n(n-1)(n-2)}{6} = \frac{n^3 - 3n^2 + 2n}{6}; \text{ and this calculation works}$$

in every case.

4. Bound: $O(n^3)$

5. Space : Auxiliary space is $O(1)$, and the result is a single int $O(1)$

Function C - Fast Exponentiation

1. Input size: Considering the number of bits needed to write it it will be

$$b = \lfloor \log_2 e \rfloor + 1$$

2. Primitive operation: $base * half * half$

3. Recurrence: $M(e)$ is the number of multiplications done. Base case is $M(0)$ which does nothing and each call has to make one recursive call on $\lfloor e / 2 \rfloor$, then if e is even it does 1 multiplication and 2 multiplication if e is odd.

$$M(0) = 0, \quad M(e) = M(\lfloor e/2 \rfloor) + \begin{cases} 1 & e \text{ even} \\ 2 & e \text{ odd} \end{cases} ;$$

If worst case that every branch is odd then each call will half the exponent,

function to unroll the recurrence

$$M(e) \leq 2 + 2 + \dots + 2 \text{ (} b \text{ times)} = 2b = 2(\lfloor \log_2 e \rfloor + 1)$$

4. Bound: $O(\log e)$
5. Space: Since it's not an array but a recursion which uses call stack. Auxiliary space is $O(\log e) = O(b)$

Function D: Rectangle matrix multiplication

1. Input size: r , s , and c
2. Primitive operation: $first[row][k] * second[k][column]$

3. Cost function: It's nested loop which runs $r, c,$ and s times

$$T(r, s, c) = \sum_{\text{row}=0}^{r-1} \sum_{\text{column}=0}^{c-1} \sum_{k=0}^{s-1} 1 = rsc$$

4. Bound: $O(n^3)$; $n = r = s = c$

5. Space: Auxiliary space is $O(1)$ and result space is $\Theta(rc)$ because the **product** matrix was returned to the caller.

Exercise 2

Part A - Single Run Experiment timing table

n	q	Direct	Prefix
500	500	783167	18250
1000	1000	958625	39042
2000	2000	5125291	65250
4000	4000	8431250	151375
8000	8000	33795375	328875

- This table shows that direct time grows far fast than the input, approx. for every 16x change in input gives us 43x growth in direct time, while prefix is showing signs of linear growth because it roughly doubles at each input.

Part B - Repeated Experiment timing table

n	q	Direct	Prefix
500	500	105916	17225
1000	1000	460908	30008
2000	2000	2024058	65641
4000	4000	8521850	56391
8000	8000	32866358	49133

- The growth factor every time n doubles the direct method becomes consistent but this repetition was less effective for the prefix method.